Quick Sort

Quick sort is a highly efficient, divide and conquer based sorting algorithm. It works by selecting a pivot element from the array and ~~partioning~~ partitioning the other elements into two sub-arrays. One with elements less than the pivot and one with element greater than the pivot - and then recursively sorting the sub-array.

→ Algorithm :-

① Choose a pivot element [usually the last element of the array].

② Partion the array such that :

- All elements smaller than the pivot are on the ~~right~~. left.
- All elements greater than the pivot are on the right.

③ Recursively apply the same process to the left and right sub-arrays.

Akash

→ C program for Quick sort;

```
#include<stdio.h>

// function to swap two elements

void swap(int *a, int *b)
{
        int temp;

        temp = *a;
        *a = *b;
        *b = temp;
}

// partition function

int partition (int arr[], int low, int high)
{
        int i, j, pivot;

        pivot = arr[high]; // pivot element
        i = low - 1;

    for( j = low; j < high; j++)
    {
        if( arr[j] <= pivot)
        {
```

Akash

```
        i++;
        swap(&arr[i], &arr[j]);
    }
}

    swap(&arr[i+1], &arr[high]);
    return (i+1);
}

// Quicksort function

void quicksort(int arr[], int low, int high)
  {
      if(low<high)
    {
        // partion index
      int pi;

        pi = partition(arr,) low, high);

      // recursively sort elements before partion

      quicksort(arr, low, pi-1);

      // recursively sort element after partion

      quicksort(arr, pi+1, high);

    }
}
```

Akash

```
// main function

int main()
{
    int arr[10], i, n;

    printf("enter size of an array \n");
    scanf("%d",&n);

    for(i=0;i<n; i++)
    {
        printf("enter data \n");
        scanf("%d",&arr[i]);
    }

    // Quicksort function call

    quicksort(arr,0,n-1);

    printf("sorted array : ");
    for(i=0;i<n;i++)
    {
        printf("%d ",arr[i]);
    return 0;
}
```

Akash

Example :-

a[0]	a[1]	a[2]	a[3]	a[4]	a[5]
7	2	1	6	8	5 → Pivot
i	j				

low = 0

high = n-1 = 5 [n = 6]

// partition function

pivot = arr[high]

pivot = 5

i = low - 1

i = -1

j = low

j = 0

if(a[j] <= pivot)

if [7 <= 5) ✗ false

j++

j = 1

if [a[1] <= pivot)

if (2 <= 5) ✓ true

i++

i.e i = 0

Akash

swap (&a[i] , &arr[j])

swap (7, 2)

New array looks like

2	7	1	6	8	5 → pivot
	i	j			

j++

j = 2

if (2 <= 5) ✓.

i++

i.e, i = 1

swap (7, 1)

New array looks like

2	1	7	6	8	5 → pivot
			j	J	

j++;

j = 3

if (6 <= 5) ✗

j++

Akash

if(8 <= 5) ✗

swap(&a[i+1], &arr[high])

swap(&a[2], &arr[5])

swap(7, 5)

New array looks like

2	1	5	6	8	7

2	1	5	6	8	7

solve recursively before partition

solve recursively after partition

2 | 1 → pivot	6 | 8 | 7 → pivot
if(2 <= 1) ✗	if (6 <= 7)
swap (2, 1)	i++;
1 | 2	swap (6, 6)
	if(8 <= 7) ✗
	swap (8, 7)
	6 | 7 | 8

Akash

Time Complexity :-

Cases-	Time Complexity
Best	$O(n \log n)$
Average	$O(n \log n)$
Worst	$O(n^2)$

Best → The pivot splits the array into two nearly equal halves at every step.

Average → Random pivots generally give good splits.

Worst → ~~th~~ Happens when the pivot is the smallest or largest element every time.

Akash

Akash